

Including structural factors into the metrics-based code smells detection

Bartosz Walter
Faculty of Computing

Poznań University of Technology
Poznań, Poland

bartosz.walter@cs.put.poznan.pl

Błażej Matuszyk
Faculty of Computing

Poznań University of Technology
Poznań, Poland

blazej.matuszyk@gmail.com

Francesca Arcelli Fontana

University of Milano-Bicocca
Milano, Italy

arcelli@disco.unimib.it

ABSTRACT

Code smells help to discover and describe deeper problems in software design. Several automated methods of smell detection are based the analysis of a combination of code-related metrics relevant for a given flaw. However, some smells reflect more complex issues and require a holistic perspective that would cover a number of different sources of data. In this paper we experimentally verify the usefulness of including structural factors into a metrics-based detection of God Class and Brain Class code smells.

Categories and Subject Descriptors

• Software and its engineering • Software creation and management • Software post-development issues • Maintaining software

Keywords

Code smell detection; detection strategies

1. INTRODUCTION

Code smells receive much attention both from the academia and industry as a useful metaphor of code mis-design symptoms. Initially, they were supposed to directly reflect the subjective developer's perception of the code quality. Recently, numerous research studies investigate methods of automated quality assessment based on code smell identification and detection.

Currently, the existing approaches to smell detection rarely utilize information other than metrics. They are sufficient in many cases, but also have significant limitations: for example, they ignore knowledge of the code structure, about the relations between code smells, results of dynamic analysis or the history of the code module extracted from code repository. In some cases this deficiency is addressed by an ad-hoc metric that provides the missing data in quasi-quantifiable way.

For example, coupling dispersion is a metric used in detection strategies for Intensive Coupling and Dispersed Coupling code smells. It is defined as the „number of classes in which operations called from the measured operation are defined, divided by the coupling intensity”, where coupling intensity is „the number of distinct operations called by the measured operation” [1]. This metric is intended to capture the extent to which coupled methods are spread among unrelated classes in different parts of the source code. However, this information is not accurately reflected by metric, because Coupling Dispersion is defined by only the "number of classes", and it ignores the relationships between these classes. Technically, the metrics could be enhanced so that it also includes the structural data, but it would negatively impact its readability and increase the computational complexity.

The drawbacks of the purely metrics-based approach can be addressed by combining the metrics with the information coming from other sources, without transforming it to metrics. In this paper we present the concept of including the structural factors to a smell detector in order to improve the detection accuracy. This approach is demonstrated and validated on God Class and Brain Class code smells with a prototypic tool of our design and implementation.

The paper is organized as follows: in Section II we make an overview of two code smells examined in this study and their detection methods; in Section III we introduce the tool used for the analysis; Section IV describes the validation procedure and its outcome; Section V presents the related work. Finally, in Section VI we conclude and outline some future research directions.

2. CODE SMELL DETECTION METHODS

Most of the currently available smell detectors are based on one of two approaches: metrics-based that employs popular code metrics, and the analysis of code graphs and their similarities. Foundations, motivation and the results for these two methods are very different.

The metrics-based approach was popularized by Marinescu [1], who introduced detection strategies. They are „quantifiable expressions of a rule by which design fragments that are conforming to that rule can be detected in the source code” [2], and the quantifiable expressions combine relevant design metrics and experimentally determined threshold values. Currently, this approach seems to dominate among other detection methods both in research and practice.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

XP 2015 Workshops, May 25-29, 2015, Helsinki, Finland

© 2015 ACM. ISBN 978-1-4503-3409-9/15/05...\$15.00

DOI: <http://dx.doi.org/10.1145/2764979.2764990>

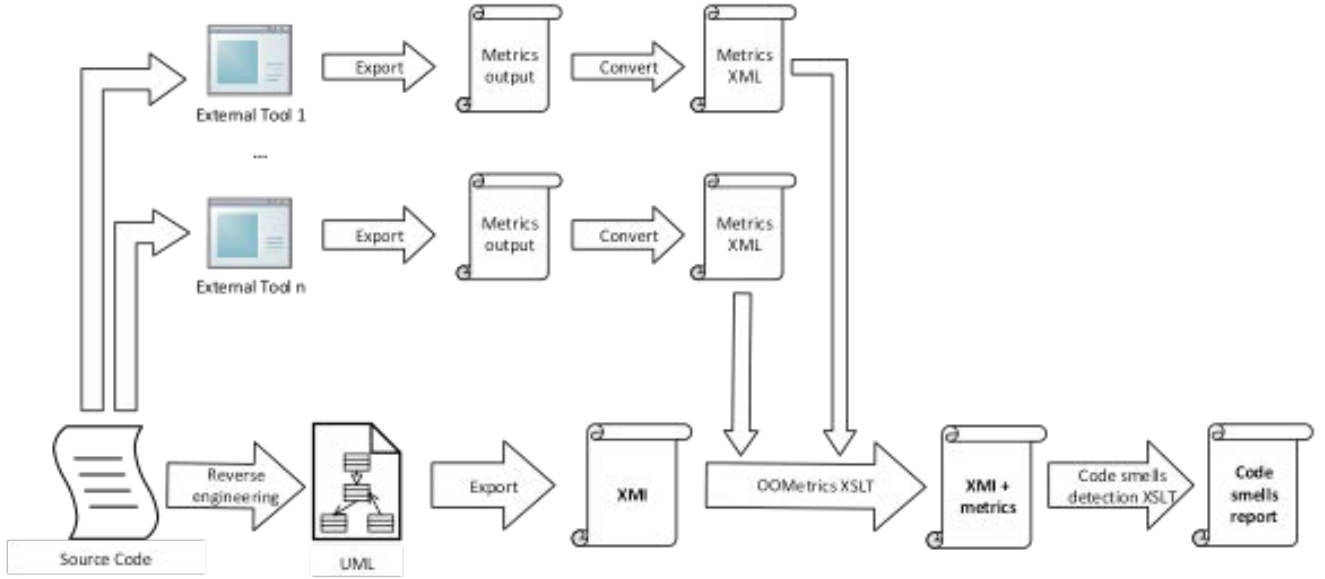


Figure 1 Overview of the analysis process in Code Navigator.

The structural approach is based on analysis of the graph representing the code, and its similarity to the pre-defined graphs describing some kinds of design violations. Like the metrics-based approach, this one also does not embrace some data and is limited to the inter-class dependencies, and, consequently, inter-class smells.

Both approaches have limitations which make them insensitive or over-sensitive to specific features of some smells. Therefore, in [14] we proposed a holistic model of code smell detection that takes into account data of different origin and scope. In this study we analyze the combination of metrics and structural factors. For that purpose we defined extended detection strategies for two smells: God Class and Brain Class, that are enhanced with structural factors.

2.1 God Class

The two analyzed smells originate from the more general Large Class smell [3] and describe different aspects of excessive complexity of a class.

God Class describes classes that „tend to centralize the intelligence of the system. An instance of a God Class performs most of the work, delegating only minor details to a set of trivial classes and using the data from other classes“ [4]. Marinescu proposed a detection strategy GC_c for that smell based on standard complexity metrics: WMC, TCC and ATFD¹.

$$GC_c(C) = \begin{cases} true : WMC(C) \geq 65 \wedge TCC(C) \leq 0.75 \wedge ATFD(C) > 7 \\ false : otherwise \end{cases}$$

Noticeably, the formula concentrates on the complexity measures and does not include the external Data Classes mentioned in the definition. For that reason we defined an extended detection strategy for that smell (GCE):

$$GCE(C) = \begin{cases} true : \begin{aligned} &(WMC(C) \geq 65) \wedge (WMC(C)/NOM(C) \geq 1.5) \\ &\wedge (((count(\{c \in CP(C) \wedge DC(c)\}) > 2) \wedge (ATFD(C) \geq 3)) \\ &\vee (count(\{c \in CP(C) \wedge DC(c)\}) > 3)) \\ false : otherwise \end{aligned} \end{cases}$$

where $CP(C)$ identifies the classes coupled to C and $DC(C)$ determines if C is a Data Class instance

2.2 Brain Class

Brain Class shares common roots with God Class, as it also features excessive complexity and centralization of the functionality [1]. However, unlike God Class, it does not reference foreign classes, which is reflected in its detection strategy BC_c .

$$BC_c(C) = \begin{cases} true : \begin{aligned} &\neg GC_c(C) \wedge WMC(C) \geq 47 \wedge TCC(C) < 0.18 \\ &\wedge (((NBM(C) > 1) \wedge LOC(C) \geq 134)) \\ &\vee ((NBM(C) = 1) \wedge (LOC(C) \geq 268) \wedge (WMC(C) \geq 94))) \\ false : otherwise \end{aligned} \end{cases}$$

The definition of Brain Class does not include directly any structural data, except for the negation of the God Class presence. The extended detection strategy BCE that references the strategy for GCE is presented below:

$$BCE(C) = \begin{cases} true : \neg GCE(C) \wedge WMC(C) \geq 65 \wedge WMC(C)/NOM(C) \geq 1.5 \\ false : otherwise \end{cases}$$

3. CODE NAVIGATOR

In order to validate the presented idea of combined detection of code smells, we needed an appropriate tool. Code Navigator is a prototypic application that provides the means to navigate and query the structure object-oriented models, which can be combined with several other data sources. At the moment, it uses only metrics and the code structure, but other data sources can be added in a similar way. As the tool name suggests, the capability of code navigation is the core feature of the system (Fig. 1).

The analysis is conducted in several subsequent steps. First, the high-level model of the code internal structure is built. This

¹ Threshold values were adjusted to reflect the expert evaluation more accurately, as described in section 4.1.

step transforms the code to the language-independent model that includes classes, their relationships and properties. As a consequence, next steps do not rely on the specific features of the object-oriented programming language.

Then, the model is exported to XMI format, which is an Object Management Group standard for representing objects and their relations in XML. This step is necessary as XML itself is not object-oriented – it can only define nested entities and their attributes, but not objects. XMI bridges the gap by providing a standard mapping from objects defined by Unified Modeling Language to XML. XMI is commonly used to store models expressed in UML [5].

XMI models offer only the structural perspective of the code and do not include the code-level data (like metrics). For that reason, in the next step the XMI model is supplemented with such additional data by applying an appropriate XSLT transformation on it. XSLT is an XML-based language for transforming XML documents into other XML documents or other formats [6]. In this case, the transformation just adds *xmi:Extension* nodes and the remaining part of the model is preserved as it is. These nodes, which represent additional data intended to be used in smell detection process, are stored inside respective XMI elements that correspond to operations, classes, packages and entire system.

Code Navigator supports two methods of acquiring the additional data: internal and external, depending on the availability of data. The internal method exploits the XSLT capabilities to compute the data from the model by using functions and templates defined in the stylesheet. This approach reduces the number of external tools, but in many cases also increases the computational complexity. This method can be applied only to data which is already available at the class- and package levels.

The external method relies on the data provided by external tools and injected to the XMI as *xmi:Extension* nodes. This allows for including data that could not be otherwise computed within the XMI model, like code-level metrics, data about history of the artifact, relations with other smells etc.

Finally, the XMI model is processed with another XSLT stylesheet, which acts as a filter to select the candidate methods, classes or packages that meet the criteria defining the given code smell. Since the stylesheet is easily extensible and modifiable, code smells definitions can be added or modified.

4. VALIDATION

In order to evaluate the proposed method, we implemented four detection strategies, two for both God Class and Brain Class code smells: a classic one proposed by Marinescu [1], and the extended one that also exploits the structural data. The experiment has been conducted on selected parts of the Apache Tomcat 7.0.14 source code.

4.1 Calibration

In the beginning we calibrated the thresholds in detection strategies. Six experts (post-grad students, having diverse, yet considerable practical knowledge of code smells) reviewed a fragment of the subject code consisting of about 100 classes. They were asked to manually identify God Classes and Brain Classes, based on their personal experience. Since programmers display different sensitivity to code smells, the results were not uniform (Fleiss kappa $\kappa_{GC}=0.306$ for God Class and $\kappa_{BC}=0.093$ for Brain Class). In order to extract representative candidates for code

smells, for further analysis we selected 100 instances indicated by at least three experts.

4.2 Comparison of classic and extended detection strategies

The results of calibration have been used for calculating the metric thresholds in classic and extended strategies.

The results for God Class are presented in Table 1.

Classic detection strategy				Extended detection strategy			
FP%	FN%	recall	precision	FP%	FN%	recall	precision
1.2	30.4	56.3	92.1	1.2	5.9	93.8	98.0

Table 2 Comparison of classic and extended detection strategies for God Class.

FP% and FN% columns present false-positive ratio and false-negative ratio, respectively. It is visible that the extended detection strategy has a lower false-negative rate. Numbers of falsely-positive instances in both cases are comparable and small (although they can be relatively easily removed manually by a manual review). The precision is also slightly better for the extended strategy.

The results for Brain Class are presented in Table 2. In this case the extended detection strategy also performs better than the "classic" one, but the difference is smaller than in case of God Class detection strategies.

For both smells we observe significant increase in the recall, which allows for concluding that the extended strategies demonstrate better performance to identify the smell instances.

Classic detection strategy				Extended detection strategy			
FP%	FN%	recall	precision	FP%	FN%	recall	precision
0	47.1	11.1	92.1	4.2	18.2	77.8	94.1

Table 3 Comparison of classic and extended detection strategies for Brain Class.

4.3 Extrapolation of results in a larger sample

Finally, we applied both types of detection strategies to the remaining parts of the subject code (consisting of ca. 900 classes) to verify whether the ratio of the precision for extended detection strategies related to the "classic" ones remains similar to the ratio calculated from the expert-reviewed sample.

In this part, we assumed that the results provided by the extended detection strategy presented in Tables 1 and 2 are completely correct. Next, the number of smell instances were divided by the number detected by the classic strategy. The recall and precision of the classic strategy relative to the extended detection strategy for both smells are presented in Tables 3 and 4, respectively. For a reference, the relative accuracy achieved in the first part of the experiment is also provided (the *Training set* column).

Test set				Training set			
FP%	FN%	recall	precision	FP%	FN%	recall	precision
0.9	39.5	34.8	95.7	1.2	30.4	56.3	92.1

Table 4 Recall and precision of FP and FN in the extended strategy for God Class.

Test set				Training set			
FP%	FN%	recall	precision	FP%	FN%	recall	precision
0.3	48.6	5.5	92.0	0	47.1	11.1	92.1

Table 5 Recall and precision of FP and FN in the extended strategy for Brain Class.

The results show that the relative performance of the extended design strategies remains fairly coherent for both the expert-reviewed, small sample, and the remaining part of the subject code.

5. THREATS TO VALIDITY

This section summarizes several factors that can threaten the validity of the presented results.

Internal validity In this study we analyzed a relatively small fragment of source code. Therefore, the accuracy of the extended detection strategy was demonstrated only for this code. The analysis performed in the second part of experiment are just extrapolation, and there is no confirmed information about code smells for the remaining, larger part of code

External validity The analysis has been conducted on a single project only. It is possible that extended detection strategies will not perform so well when applied to the of systems of of different domains, different size or displaying other distinctive features. Also, since the detection strategies were calibrated on a small sample of specific code, the relation between the classic and extended strategies may vary for other systems.

6. RELATED WORK

Several approaches and tools for code smell detection have been proposed so far in the literature. Some of them have been developed to improve code quality during software development, while other support reengineering and maintenance activities.

Most of the available tools are not able to refactor the code based on detected smells, but modern IDEs are usually capable of performing refactoring automatically, also if they need the user's guidance.

It would be desirable for code smell detectors to help the user to understand the cause of the reported symptom and, as outlined by Murphy-Hill and al. [8], the tools should also not display smell information in a way that overloads the programmer, in case of smell proliferation.

The code smell detection rules are usually based on:

- Computation of a set of metrics which have to satisfy some conditions;
- Program analysis aimed at identifying the refactoring opportunities that would remove the smell;
- Machine learning techniques;
- Dedicated specification language.

The majority of the available tools use metrics-based techniques, e.g. inFusion, Checkstyle, PMD, iPlasma, and CodeVizard. They often provide different results due to differences in the rules, selected metrics and thresholds. A single rule can involve only one metric, as for example the number of LOC metric in the case of Large Class in PMD, or several different metrics for the detection of God Class (similar to Large Class) in inFusion and iPlasma.

Similar approach is presented in the DETEX method [9]. It uses a dedicated specification language to define and detect code and design smells at high level of abstraction, using a unified vocabulary and a domain-specific language.

JDeodorant is an example of the structure-based smell detectors [10]. It computes and compares similarity scores

between the smell instance and smell candidate, and also proposes refactoring opportunities to remove the smell.

Machine learning is another concept used for smell detection. Yang et al. [11] study the judgment of individual users by applying machine learning algorithms to the detected code clones. Arcelli et al. [12] exploit different machine learning algorithms for the detection of four code smells. Maiga et al. [13] introduce SVMDetect, which is based on the support vector machines.

7. CONCLUDING REMARKS AND FUTURE WORKS

In this paper we presented a method of code smell detection based on a combination of metrics and the structural data about the analyzed code. It is also potentially capable of integrating other data that is appropriate for detecting code smells more accurately. Moreover, we also developed a prototypic tool in order to validate the approach. Its application to a sample code delivered evidence that the approach can be more accurate in the detection proces.

However, the method has also weaknesses. The most important concern is performance. Computational complexity of calculating majority of metrics, as well as applying the detection strategies, is at least $O(n^2)$. Therefore, in case of very large projects consisting of tens of thousands of classes, this method may be considered inefficient. Also the technology chosen for implementing the prototype can be questioned. Although metric definitions and detection strategies are relatively easy to implement and maintain in XSLT, their execution in an interpreted environment may not be as efficient as it would be if they were natively compiled.

In order to evaluate the method, we conducted an experiment which compares the accuracy of classic detection strategies with the accuracy of extended strategies proposed in the paper. The results allowed for concluding that the detection strategies extended with structural information perform better in terms of recall, at a comparable precision level.

In the future we plan to extend the tool with other data sources and implement detection strategies for other class-level code smells, like Feature Envy or Inappropriate Intimacy. A planned large-scale experiment will also deliver more evidence concerning the method's advantages and disadvantages.

8. REFERENCES

- [1] M. Lanza and R. Marinescu: Object-Oriented Metrics in Practice: Using Software Metrics to Characterize, Evaluate, and Improve the Design of Object-Oriented Systems. Springer, 2006.
- [2] R. Marinescu: Detection strategies: Metrics-based rules for detecting design flaws. In: Proc. of ICSM 2004, pp. 350-359.
- [3] M. Fowler: Refactoring: Improving the Design of Existing Code. Addison-Wesley Professional, 1999.
- [4] R. Marinescu: Measurement and Quality in Object-Oriented Design. PhD thesis, Politehnica University of Timisoara, 2002
- [5] T.J. Grose, G.C. Doney, and S.A. Brodsky: Mastering XMI: Java Programming with XMI, XML, and UML. Wiley, 2002.
- [6] D. Tidwell: XSLT, 2nd Edition. O'Reilly Media, 2008.
- [7] S.M. Olbrich, D.S. Cruzes, and D.I.K. Sjoberg: Are all code smells harmful? A study of God Classes and Brain Classes in the evolution of three open source systems. In: Proc. of ICSM 2010, pp.1-10.

- [8] E. Murphy-Hill and A. P. Black. An interactive ambient visualization for code smells. In: Proc. of the 5th ISSV SOFTVIS '10, Salt Lake City, Utah, USA, 2010, pp. 5-14.
- [9] N. Moha, Y.-G. Guéhéneuc, L. Duchien, and A.-F. Le Meur, DECOR: A Method for the Specification and Detection of Code and Design Smells, IEEE TSE, vol. 36, no. 1, pp. 20-36.
- [10] JDeodorant Homepage. <http://www.jdeodorant.com> (visited in March 2015)
- [11] J. Yang, K. Hotta, Y. Higo, H. Igaki, and S. Kusumoto: Filtering clones for individual user based on machine learning analysis. In: Proc. of the 6th IWSC 2012, Zurich, Switzerland. IEEE Computer Society, June 2012, pp. 76-77.
- [12] F. Arcelli Fontana, M. Mäntylä, A. Marino, and M. Zanoni: Code Smell Detection: towards a Machine Learning-based Approach. In: Proc. of ICSM 2013 (ERA Track), 2013.
- [13] A. Maiga, N. Ali, N. Bhattacharya, A. Saban'e, Y.-G. Gu'h'eneuc, G. Antoniol, and E. A'imeur, Support vector machines for antipattern detection. In: Proc. of the 27th IEEE/ACM ASE 2012. Essen, Germany: ACM, 2012, pp. 278-281.
- [14] B. Walter, and B. Pietrzak: Multi-criteria detection of bad smells in code with UTA method. In: Extreme Programming and Agile Processes in Software Engineering, Springer, 2005, pp. 154-161.